CHAPTER

TWELVE

THREAD ADVANCE

12.1 Acknowledgement

The examples used in this chapter are based on examples in CUDA BY EXAMPLE: An Introduction to General-Purpose GPU Programming, written by Jason Sanders and Edward Kandrot, and published by Addison Wesley.

12.2 Vector Dot Product

In this example we will see how to perform a dot product using GPU computation. We know that the result of vector addition is a vector, but the result of vector dot product is a number. However, we can divide the vector dot product process into two steps. We first use CUDA to the multiplication process. After this step, the device will return a vector with all its elements as multiplication results to the host code. Then the CPU can do all the adding up process.

Vector Dot Product source file: `Dot-GM.cu`

12.2.1 The Device Code

```
__global__ void dot( float *a, float *b, float *c ) {

    // keep track of the index
    int tid = threadIdx.x + blockIdx.x * numThread;
```

```
    while (tid < N) {
        c[tid] = a[tid] * b[tid];
        tid += numThread * numBlock; // shift by the total number of thread in a grid
    }
}
```

The device code is pretty straight forward. Each thread multiplies a pair of corresponding elements in two vectors. After each thread done their job for the first time, if there are still elements left unprocessed, they runtime will instruct the threads to do another round of computation until all the elements are processed.

12.2.2 The Host Code

```
int main( void ) {

    float   *a, *b, sum, *c;
    float   *dev_a, *dev_b, *dev_c;

    // allocate memory on the cpu side
    a = (float*)malloc( N*sizeof(float) );
    b = (float*)malloc( N*sizeof(float) );
    c = (float*)malloc( N*sizeof(float) );

    // fill in the host memory with data
    for (int i=0; i<N; i++) {
        a[i] = i;
        b[i] = i*2;
    }

    // start the timer
    cudaEvent_t     start, stop;
    HANDLE_ERROR( cudaEventCreate( &start ) );
    HANDLE_ERROR( cudaEventCreate( &stop ) );
    HANDLE_ERROR( cudaEventRecord( start, 0 ) );

    // allocate the memory on the GPU
    HANDLE_ERROR( cudaMalloc( (void**)&dev_a, N*sizeof(float) ) );
    HANDLE_ERROR( cudaMalloc( (void**)&dev_b, N*sizeof(float) ) );
    HANDLE_ERROR( cudaMalloc( (void**)&dev_c, N*sizeof(float) ) );

    // copy the arrays 'a' and 'b' to the GPU
    HANDLE_ERROR( cudaMemcpy( dev_a, a, N*sizeof(float), cudaMemcpyHostToDevice ) );
    HANDLE_ERROR( cudaMemcpy( dev_b, b, N*sizeof(float), cudaMemcpyHostToDevice ) );
```

```
    dot<<<numBlock,numThread>>>( dev_a, dev_b, dev_c );

    // copy the array 'c' back from the GPU to the CPU
    HANDLE_ERROR( cudaMemcpy( c, dev_c, N*sizeof(float), cudaMemcpyDeviceToHost ) );

    // get stop time, and display the timing results
    HANDLE_ERROR( cudaEventRecord( stop, 0 ) );
    HANDLE_ERROR( cudaEventSynchronize( stop ) );
    float   elapsedTime;
    HANDLE_ERROR( cudaEventElapsedTime( &elapsedTime,
                                        start, stop ) );
    printf( "Time to generate:  %3.1f ms\n", elapsedTime );

    // finish up on the CPU side
    sum = 0;
    for (int i=0; i<N; i++) {
        sum += c[i];
    }

    #define sum_squares(x)  (x*(x+1)*(2*x+1)/6)
    printf( "Does GPU value %.6g = %.6g?\n", sum,
             2 * sum_squares( (float)(N - 1) ) );

    // free memory on the gpu side
    HANDLE_ERROR( cudaFree( dev_a ) );
    HANDLE_ERROR( cudaFree( dev_b ) );
    HANDLE_ERROR( cudaFree( dev_c ) );

    // free memory on the cpu side
    free( a );
    free( b );
    free( c );
}
```

The host code the much like the vector addition example. We first allocate the memory on host memory and device memory. Then we initialize the matrices and fill them with data. After that we copy the data from host to device and execute the kernel code. Finally we transfer the data back from device memory to host memory. Do not forget to use Event API to measure the performance.

However, we need to point out two differences.

```
float   *a, *b, sum, *c;
float   *dev_a, *dev_b, *dev_c;
```

First is that we need to declare one more pointer for the host code. In the vector addition example, two sets of array pointers are enough. However, in this example, we are returning a number. Therefore, a pointer which pointing to that number is essential.

```
// finish up on the CPU side
sum = 0;
for (int i=0; i<N; i++) {
    sum += c[i];
}
```

Another difference is that we need to add up all the elements in the returned vector. This can simply be done by adding a for loop in the host code. Notice that we put this loop outside of the Event API so that it will not interfere our timing result.

You can verify the result by adding the following code into the host code.

```
#define sum_squares(x)  (x*(x+1)*(2*x+1)/6)
printf( "Does GPU value %.6g = %.6g?\n", sum,
         2 * sum_squares( (float)(N - 1) ) );
```

For people who are familiar with discrete math, the code above should be simple to apprehend. This function will give the result through a more *clever* way.

12.3 Vector Dot Product with Reduction

In the previous example, you might have the question: why do we need to return the whole array back to the CPU? Is is possible for use to first reduce them a little bit and then return it to the CPU?

Well, we can do reduction in CUDA. In this chapter, we will see how to use reduction in CUDA. But before we proceed, we first need to know something about shared memory in CUDA. In each of the blocks we create, CUDA runtime will assign a region memory to this block. This type of memory is called shared memory. When you are declaring your variables, you can add the CUDA C keyword **__shared__** to make this variable reside in shared memory. Why do we need shared memory?

When we are learning Block and Thread, we had the question why would we need two hierarchy to organize threads question in mind. Well, part of the reason is that we can benefit from shared memory by organize threads in blocks. When we declare a variable and make it reside in shared memory, CUDA runtime creates a copy of this variable in each block you launched in host code. Every threads in one block shares this memory, which means they can see and modify their shared memory. However, they cannot see or modify shared memory assigned in other blocks. What does this mean to us? Well, if you can have one region of memory that is private to threads in one block, you can explore ways to facilitate communication and collaboration between threads within this block.

Another reason we like to use shared memory is that it is faster than global memory we used to use. As the latency of shared memory tends to be far lower than global memory, it is the ideal choice to serve as cache in each block.

In this example, we will see how we use shared memory to serve as a cache-per-block and how we can perform reduction on it.

Vector Dot Product with Reduction source file: `Dot-SM.cu`

12.3.1 The Device Code

```
__global__ void dot( float *a, float *b, float *c ) {

    // declare cache in the shared memory
    __shared__ float cache[numThread];

    // keep track of thread index
    int tid = threadIdx.x + blockIdx.x * numThread;
    // connect thread index and cache index
    int cacheIndex = threadIdx.x;

    float   temp = 0;
    while (tid < N) {
        temp += a[tid] * b[tid];
        tid += numThread * numBlock;// increase by the total number of thread in a grid
    }

    // set the cache values
    cache[cacheIndex] = temp;

    // synchronize threads in this block
    __syncthreads();
```

```
    // for reductions, numThread must be a power of 2 because of the following code
    int i = numThread/2;
    while (i != 0) {
        if (cacheIndex < i)
            cache[cacheIndex] += cache[cacheIndex + i];
        __syncthreads();
        i /= 2;
    }

    // write back to the global memory
    if (cacheIndex == 0)
        c[blockIdx.x] = cache[0];
}

    // declare cache in the shared memory
    __shared__ float cache[numThread];
```

In the two lines of code above, we declared a cache in shared memory for this block. We will then use this cache to store each thread's running sum. You can also see that we set the size of the cache same as numThread so that each thread in the block can have its own place to store its running sum. Notice we only create one copy of cache instead of creating numBlock copies. The compiler will automatically create a copy of cache in each block's shared memory, meaning we only have to declare one copy.

```
    float   temp = 0;
    while (tid < N) {
        temp += a[tid] * b[tid];
        tid += numThread * numBlock;// increase by the total number of thread in a grid
```

The actual vector dot product computation is the similar to what we seen in the global memory version. However, there is little difference. You can see from the code above, instead of using

```
c[tid] = a[tid] * b[tid];
```

we use

```
temp += a[tid] * b[tid];
```

The reason causes this difference is that we are finding a running sum in this example. In the previous example, as we are returning a vector exactly the same size as the input, we have enough space to store each multiplication results. Even

we have less threads than vector dimension so that each thread will compute more than one value, they can store each value in different places. However, in this example, we have exactly the same amount of place for storage as number of threads in each block. If any thread compute more than once, they still have only one place to store values. This bring us to why we need to use running sum of each thread.

You may notice we add a line of code you have never seen before.

```
// synchronize threads in this block
__syncthreads();
```

We have seen code similar to this before. When we are learning how to use CUDA Event API to measure performance, we used command **cudaEventSynchronize()** to synchronize the stop event. The purpose of **__syncthreads()** is somehow similar to the command **cudaEventSynchronize()**. When we are using shared memory to facilitate communication and collaboration between threads, we need to create a way to synchronize them as well. For example, if one thread is writing a number to the shared memory and another thread need to use that number for further computation, we want the first thread to finish its work before the second thread executes its command. What the command __syncthread() will do, is essentially create a barrier for all the threads and block them from executing further command. After all threads have finished executing commands before __syncthread(), then they can all proceed to the next command.

After we make sure all the elements in cache is filled, we can proceed to the reduction process.

```
// for reductions, numThread must be a power of 2 because of the following code
int i = numThread/2;
while (i != 0) {
    if (cacheIndex < i)
        cache[cacheIndex] += cache[cacheIndex + i];
    __syncthreads();
    i /= 2;
}
```

Suppose we have a cache with 256 entries. What the code above would do is that it first take 128 thread in the block and each thread will add two of the values in cache[a] and cache[a+128] and store the value back to cache[a]. Then in the second iteration it will take 64 thread in the block and each thread will add values in cache[a] and cache[a+64] and store the value back to cache[a]. After log2(numThread) times of operation, we would have the sum of all 256 values stored in the first element of the cache. Be really careful that we need to synchro-

nize threads every time after we perform one reduction.

Finally, we choose the thread with index 0 to write the result back to the global memory.

```
    // write back to the global memory
    if (cacheIndex == 0)
        c[blockIdx.x] = cache[0];
}
```

12.3.2 The Host Code

In general, the host code of the shared memory version is very similar to that of the global memory version. However, there are several difference we need to point out.

```
    partial_c = (float*)malloc( numBlock*sizeof(float) );
    HANDLE_ERROR( cudaMalloc( (void**)&dev_partial_c,
                              numBlock*sizeof(float) ) );
```

In the above two lines of code, in the previous example, we declared two sets of pointers with each pointer pointing to array having the same size. This time, however, we need the output array to be smaller than the vector size. Since every block's will write only one number back to the global memory, the size of output array should be numBlock.

Another point worth mentioning is that we define the numBlock in the following way instead of assign a number to it.

```
#define imin(a,b) (a<b?a:b)
const int numBlock = imin( 32, (N+numThread-1) / numThread );
```

When we are choosing the number of blocks to launch in this problem, we faces to requirements. First is that we should not create too many blocks. In the final step where all the results returned by all the blocks are summed up, we are using CPU to compute. This means if we create too many blocks, we would leave CPU too much workload. Another requirements is that we cannot assign too less blocks either. As we can only fit 256 threads in each block, if we assign not enough blocks, we would end up having each thread doing many times of computation. Facing this two requirements, we came up with the solution above. We use the smaller number between 32 and (N+numThread-1) / numThread. The function (N+numThread-1) / numThread gives the smallest multiple of numThread that is equal or larger than the vector size. Calculating this number will ensure we have just enough blocks so that each element in a small vector has its own thread.

If we are facing a small vector, we can us the later to assign not too many blocks. If we are facing a gigantic vector, 32 blocks is somehow enough to keep the GPU busy.

Be aware the number 32 was given by a CUDA programming book that is several years old. We decide to use it because we think its safe and yet sufficient for our problem size. If you are dealing with much larger problem size and have much more powerful GPU cards in hand, feel free to stretch this number to thousands even hundreds of thousand.